DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

Projet JAVA – Etape 5

Table des matières

Etape 5 : 2
Modification de « JFrameLesExcursions » 2
Création de la nouvelle interface graphique 3
Rédaction du programme 5
Dans « JFrameLesExcursions » 5
Dans « JDialogAjouter » 6
Tests fonctionnels 9
Bilan 14

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

Etape 5 :

Modification de « JFrameLesExcursions »

Dans un premier temps, on ajoute deux boutons suivants sur la « JFrame » : « Création », « Visualisation », « Modification » et « Suppression ». Pour le moment, nous allons nous intéresser uniquement au bouton « Création » comme le demande le cahier des charges.

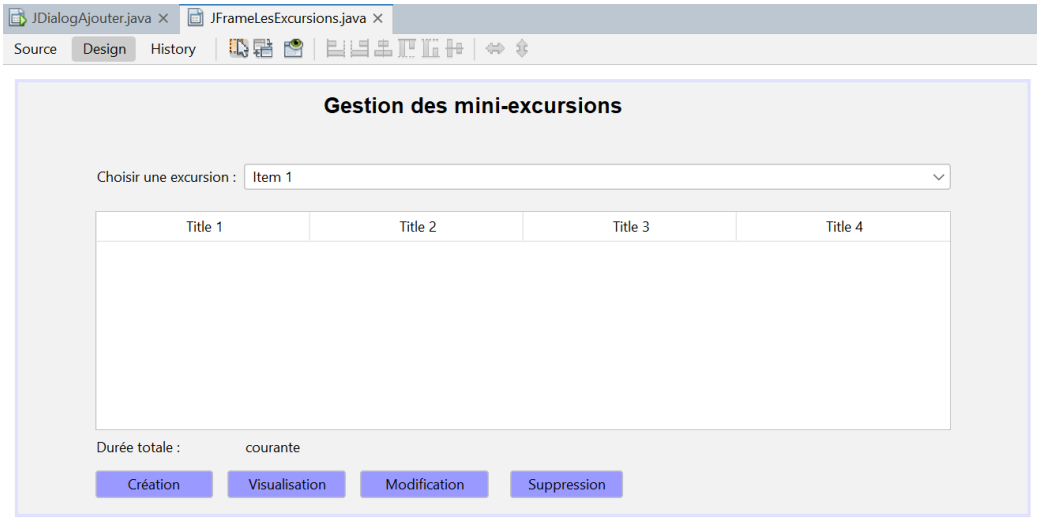

Figure 1 : JFrameLesExcursions.java : Interface graphique. Ajout de 4 nouveaux boutons (ceux en violet).

```
    // Variables declaration - do not modify
    private javax.swing.JButton jButtonCreation;
    private javax.swing.JButton jButtonLecture;
    private javax.swing.JButton jButtonModification;
    private javax.swing.JButton jButtonSupprimer;
    private javax.swing.JComboBox<String> jComboBoxExcursions;
    private javax.swing.JDialog jDialog1;
    private javax.swing.JDialog jDialog2;
    private javax.swing.JDialog jDialog3;
    private javax.swing.JDialog jDialog4;
    private javax.swing.JDialog jDialog5;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JLabel jLabelDureeTotale;
    private javax.swing.JLabel jLabelDureeTotaleCourante;
    private javax.swing.JScrollPane jScrollPane1;
    private javax.swing.JTable jTableEtapes;
    // End of variables declaration

}
```

Figure 2 : JFrameLesExcursions.java : Nom des variables présentes sur l'interface grpahique.

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

Création de la nouvelle interface graphique

Tout d'abord, on créé un nouveau « JDialog » qui va servir à ajouter de nouvelles excursions. On commence par placer en tête de fenêtre, son titre qui décrit son rôle. Puis, on ajoute 4 nouvelles lignes commençant par un « JLabel » qui explique quoi rédiger, suivi un « JTextField » que l'utilisateur pourra remplir. Enfin, le bouton « OK » sert à ajouter les informations à la base de données. Quant au bouton « Annuler », il ferme la fenêtre et les données qui ont été entrées précédemment sont supprimées.

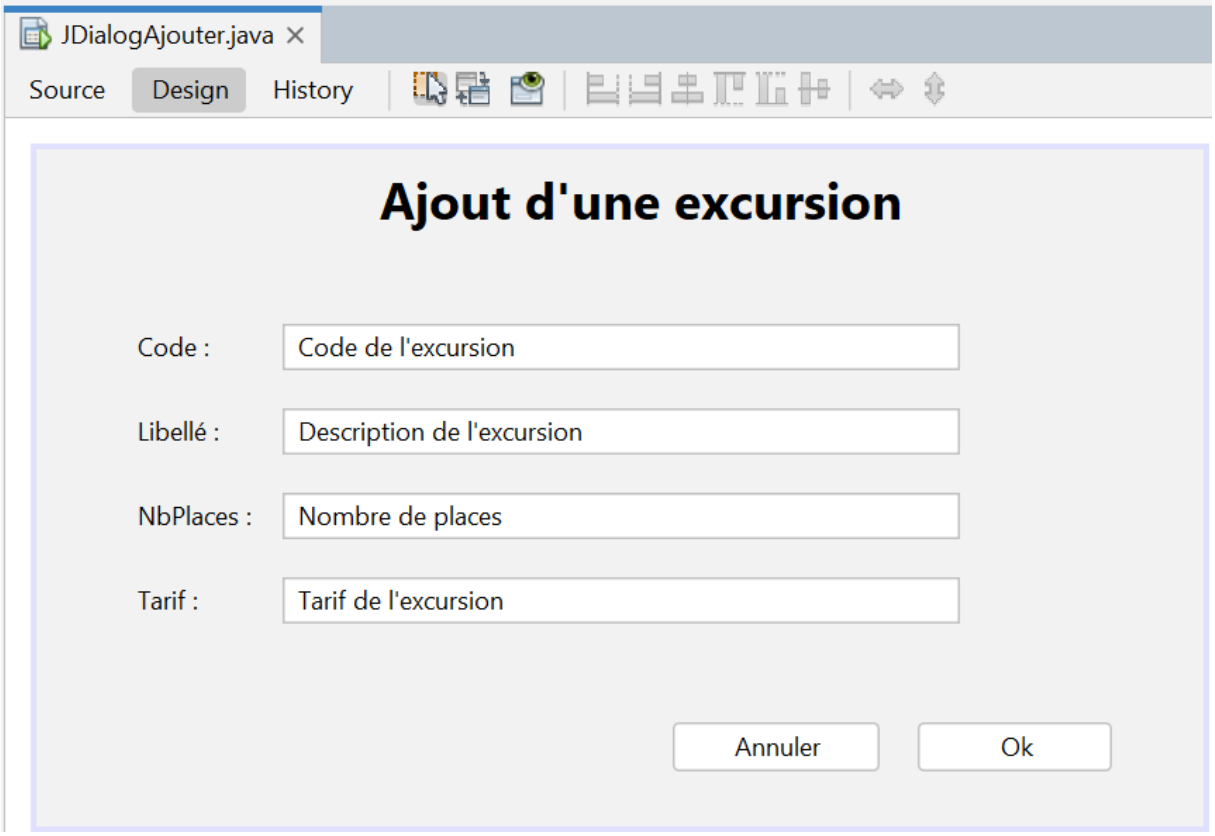

Figure 3 : JDialogAjouter.java : Interface graphique.

Les noms de ces variables sont écrits juste en dessous. Seulement, il y a un bug qu'on ne comprend pas. Lorsqu'on regarde le code on peut voir des labels très similaires comme : « jLabelCode » et « jLabelCodeAjouter ». Il semblerait que NetBeans est conservé en mémoire les anciens noms de labels. A savoir que les labels similaires ont le même contenu.

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

```
314        // Variables declaration - do not modify
315        private javax.swing.JButton jButtonAnnuler;
316        private javax.swing.JButton jButtonOk;
317        private javax.swing.JLabel jLabelCode;
318        private javax.swing.JLabel jLabelCodeAjout;
319        private javax.swing.JLabel jLabelLibelle;
320        private javax.swing.JLabel jLabelLibelleAjout;
321        private javax.swing.JLabel jLabelNbPlaces;
322        private javax.swing.JLabel jLabelNbPlacesAjout;
323        private javax.swing.JLabel jLabelTarif;
324        private javax.swing.JLabel jLabelTarifAjout;
325        private javax.swing.JLabel jLabelTitrePrincipal;
326        private javax.swing.JTextField jTextField2;
327        private javax.swing.JTextField jTextFieldCodeAjouter;
328        private javax.swing.JTextField jTextFieldLibelleAjouter;
329        private javax.swing.JTextField jTextFieldNbPlacesAjouter;
330        private javax.swing.JTextField jTextFieldTarifAjouter;
331        // End of variables declaration
```

Figure 4 : JDialogAjouter.java : Noms des variables présentes sur l'interface graphique précédente.

```
jLabelCode.setText("Code : ");

jLabelLibelle.setText("Libellé :");

jLabelNbPlaces.setText("NbPlaces :");

jLabelTarif.setText("Tarif :");

jTextField2.setText("code de l'excursion");

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);

jLabelTitrePrincipal.setFont(new java.awt.Font("Segoe UI", 1, 24)); // NOI18N
jLabelTitrePrincipal.setText("Ajout d'une excursion");
jLabelTitrePrincipal.setMinimumSize(new java.awt.Dimension(200, 30));
jLabelTitrePrincipal.setPreferredSize(new java.awt.Dimension(200, 30));

jLabelCodeAjout.setText("Code : ");

jLabelLibelleAjout.setText("Libellé :");

jLabelNbPlacesAjout.setText("NbPlaces :");

jLabelTarifAjout.setText("Tarif :");

jTextFieldCodeAjouter.setText("Code de l'excursion");
```

Figure 5 : JDialogAjouter.java : Bugs de la part de NetBeans, les anciens noms de variables sont conservés.

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

Rédaction du programme

Dans « JFrameLesExcursions »

Tout d'abord on commence par rédiger une méthode servant à rafraichir les excursions présentes dans « JComboExcursions ». On commence par initialiser une Arraylist avec toutes les excursions de la base de données. Ensuite, on supprime tous les éléments dans « JComboExcursions ». Puis, on ajoute chaque élément de l'Arraylist un à un. De ce fait, les excursions présentes dans la base de données apparaissent belles et biens dans « JComboExcursions ». Si une erreur est détectée, un POP-UP surgit avec son intitulé.

```java
121     /**
122      * Rafraichit les excursions présentes dans le JComboBox
123      *
124      */
125     public void rechargerExcursions() {
126         try {
127             ArrayList<MiniExcursion> excursions = DaoMiniExcursion.getAll(); // Sélection de tous les éléments qu'on ajoute dans une Arraylist
128             modeleJComboExcursions.removeAllElements(); // On supprime tous les éléments du JComboBox
129             // Puis on rajoute de nouveau les données
130             for (MiniExcursion ex : excursions) {
131                 modeleJComboExcursions.addElement(ex);
132             }
133         } catch (SQLException ex) {
134             // Affiche une erreur si les excursions ont un problème lors du chargement
135             JOptionPane.showMessageDialog(this, "Erreur lors du rechargement des excursions : " + ex.getMessage());
136         }
137     }
```

Figure 6 : JFrameLesExcursions : Rechargement des excursions.

Par la suite, la méthode rattachée au bouton « Creation » instancie un objet de la classe « JDialogAjouter ». Puis, il affiche la fenêtre montrée précédemment.

```java
354     /**
355      * Affiche la fenêtre reliée au bouton "Creation"
356      *
357      * @param evt
358      */
359     private void jButtonCreationActionPerformed(java.awt.event.ActionEvent evt) {
360         JDialogAjouter jDialogCreation = new JDialogAjouter(this,true);
361         jDialogCreation.setVisible(true);
362     }
```

Figure 7 : JFrameLesExcursions : Fonctionnement du bouton OK.

Enfin, le bouton « Annuler » ferme la fenêtre et ne conserve pas le contenu inscrit à l'intérieur par l'utilisateur, s'il en avait rentrée.

```java
264     /**
265      * Bouton qui ferme la fenêtre (JDialogAjouter)
266      *
267      * @param evt
268      */
    private void jButtonAnnulerActionPerformed(java.awt.event.ActionEvent evt) {
270         this.dispose(); // Ferme la fenêtre et ne conserve pas le contenu des JTextFields
271     }
```

Figure 8 : JFrameLesExcursions : Fonctionnement du bouton « Annuler ».

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

Dans « JDialogAjouter »

On commence tout d'abord par mettre à vide les « jTextFields » en modifiant leur texte grâce à la méthode « setText() ».

```
19      /**
20       * Creates new form JDialogAjouter
21       * @param parent
22       * @param modal
23       */
24      public JDialogAjouter(java.awt.Frame parent, boolean modal) {
25          super(parent, modal);
26          initComponents();
27
28          // Vide les champs de textes
29          jTextFieldCodeAjouter.setText("");
30          jTextFieldLibelleAjouter.setText("");
31          jTextFieldNbPlacesAjouter.setText("");
32          jTextFieldTarifAjouter.setText("");
33      }
```

Figure 9 : JDialogAjouter.java : Initialisation des « jTextFields » à vide.

Après ça on rédige une méthode dont l'objectif est d'insérer une excursion dans la base de données. Elle porte le nom de « insertIntoDatabase() ». Elle débute par la préparation de la requête SQL dans une variable String. Ensuite, on effectue la connexion à la base de données. Par la suite, on remplace les points d'interrogation par les attributs présents dans la signature de la méthode. Enfin, on exécute la requête SQL à la ligne 259 et on affiche un message de succès en tant que POP-UP.

```
236     /**
237      * Insertion d'une nouvelle excursion dans la BDD
238      *
239      * @param codeExcursion Identifiant de l'excursion
240      * @param libelleExcursion Titre de l'excursion
241      * @param nbPlaces Nombre de places maximales
242      * @param tarif Prix d'une excursion par personne
243      * @throws SQLException
244      */
245     private static void insertIntoDatabase(String codeExcursion, String libelleExcursion,int  nbPlaces, double tarif) throws SQLException {
246         // Préparation de la requête SQL
247         String sql = "INSERT INTO Excursion (Code, Libelle, NbPlaces, Tarif) VALUES (?, ?, ?, ?)";
248
249         // Connexion à la base de données
250         Connection connection = ConnexionBDD.getConnexion();
251         PreparedStatement pstmt = connection.prepareStatement(sql);
252
253         // On remplace chaque point d'interrogation par les informations correspodantes
254         pstmt.setString(1, codeExcursion);
255         pstmt.setString(2, libelleExcursion);
256         pstmt.setInt(3, nbPlaces);
257         pstmt.setDouble(4, tarif);
258
259         pstmt.executeUpdate(); // Exécution de la requête
260         JOptionPane.showMessageDialog(null, "Excursion ajoutée avec succès !"); // Affiche un message de succès
```

Figure 10 : JDialogAjouter.java : Méthode pour l'insertion d'une excursion dans la BDD.

En somme, on rédige une méthode « jButtonOkActionPerformed() » qui consiste en la validation et l'envoie de l'excursion que l'on souhaite ajouter. En premier lieu, on déclare deux variables qui vont stocker le nombre de places en tant qu' « int » et le tarif en « double » (nombre à virgules). Ensuite, on associe à des variables String le contenu des « JTextFields » récupérées par la méthode « getText() ». Puis, on vérifie si les champs de textes sont vides. Si c'est le cas, un message d'erreur apparaît en tant que POP-UP.

```
188     /**
189      * Bouton pour valider les informations à enregistrer
190      *
191      * @param evt
192      */
193     private void jButtonOkActionPerformed(java.awt.event.ActionEvent evt) {
194         // Déclaration de variables
195         int nbPlacesInt;
196         double tarifDouble;
197
198         // Récupère le contenu de chaque champs de texte
199         String codeExcursion = jTextFieldCodeAjouter.getText();
200         String libelleExcursion = jTextFieldLibelleAjouter.getText();
201         String nbPlacesStr = jTextFieldNbPlacesAjouter.getText();
202         String tarifStr = jTextFieldTarifAjouter.getText();
203
204         // Vérifie si l'un des champs de texte est vide
205         if (codeExcursion.isEmpty() || libelleExcursion.isEmpty() || nbPlacesStr.isEmpty() || tarifStr.isEmpty()) {
206             JOptionPane.showMessageDialog(null, "Tous les champs doivent être remplis.");
207             return;
208         }
```

Figure 11 : JDialogAjouter.java : Bouton « OK » : 1ère partie de la méthode.

Après ça, les deux premières variables qu'on avait déclaré au début de la méthode ont leur valeur convertit en le type qui leur correspond. Ensuite, on vérifie si ces deux variables ont des valeurs interdites. C'est-à-dire, « nbPlacesInt » est inférieur ou égal à 0 et « tarifDouble » est négatif. Si c'est le cas, un POP-UP apparaît et un message décrit ce qu'il faut faire. Dans le cas où des valeurs numériques ne sont pas présentes dans les bons « jTextFields », un message d'erreur apparait aussi sous la forme de POP-UP.

```
211         try {
212             // Changements de type de variables
213             nbPlacesInt = Integer.parseInt(nbPlacesStr);
214             tarifDouble = Double.parseDouble(tarifStr);
215
216             // Gère le cas d'erreur : nombre de places négatifs ou inférieur à 0 et tarif négatifs
217             if (nbPlacesInt <= 0 || tarifDouble < 0) {
218                 JOptionPane.showMessageDialog(null, "Le nombre de places et le tarif doivent être positifs.");
219                 return;
220             }
221         } catch (NumberFormatException e) {
222             // Affiche une erreur si des valeurs numériques ne sont pas présentes dans des champs de textes spécifiques
223             JOptionPane.showMessageDialog(null, "Veuillez entrer des valeurs numériques valides.");
224             return;
225         }
```

Figure 12 : JDialogAjouter.java : Bouton « OK » : 2nde partie de la méthode.

Enfin, on appelle la méthode « insertIntoDatabase() » pour insérer les données dans la base de données. Les variables déclarées précédemment sont utilisées en tant qu'attributs de cette méthode. Ensuite, on recharge les excursions avec la méthode « rechargerExcursions() ». Puis, on ferme la fenêtre. Si une erreur surgit, un POP-UP apparaît.

```
227         try {
228             insertIntoDatabase(codeExcursion, libelleExcursion, nbPlacesInt, tarifDouble); // Apppel de la méthode pour
229             ((JFrameLesExcursions) getParent()).rechargerExcursions(); // Appel de la méthode rechargerExcursions() pou
230             this.dispose(); // Ferme la fenêtre
231         } catch (SQLException ex) {
232             JOptionPane.showMessageDialog(null, "Erreur lors de l'ajout dans la base de données : " + ex.getMessage());
233         }
234     }
```

Figure 13 : JDialogAjouter.java : Bouton « OK » : 3ème et dernière partie de la méthode.

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

Tests fonctionnels

Tout d'abord, l'utilisateur lance l'application depuis NetBeans. Puis, il clique sur le bouton « Creation ».

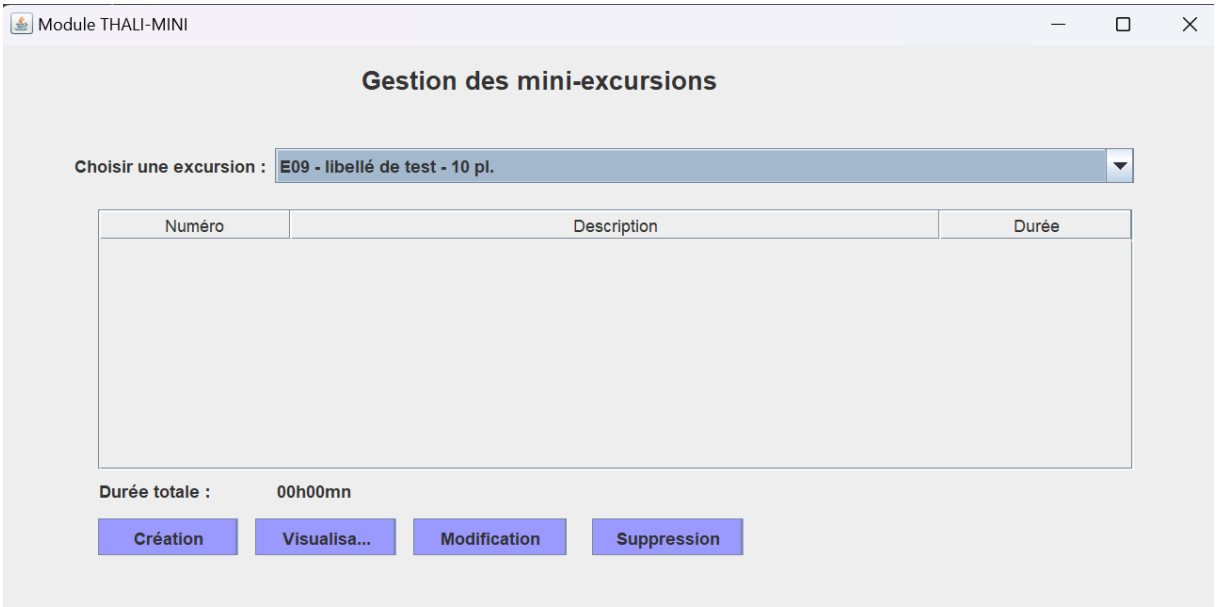

Figure 14 : Test n°1 : Fenêtre principale.

Une fois ceci fait, une excursion est ajoutée la fenêtre suivante apparaît sous la forme suivante.

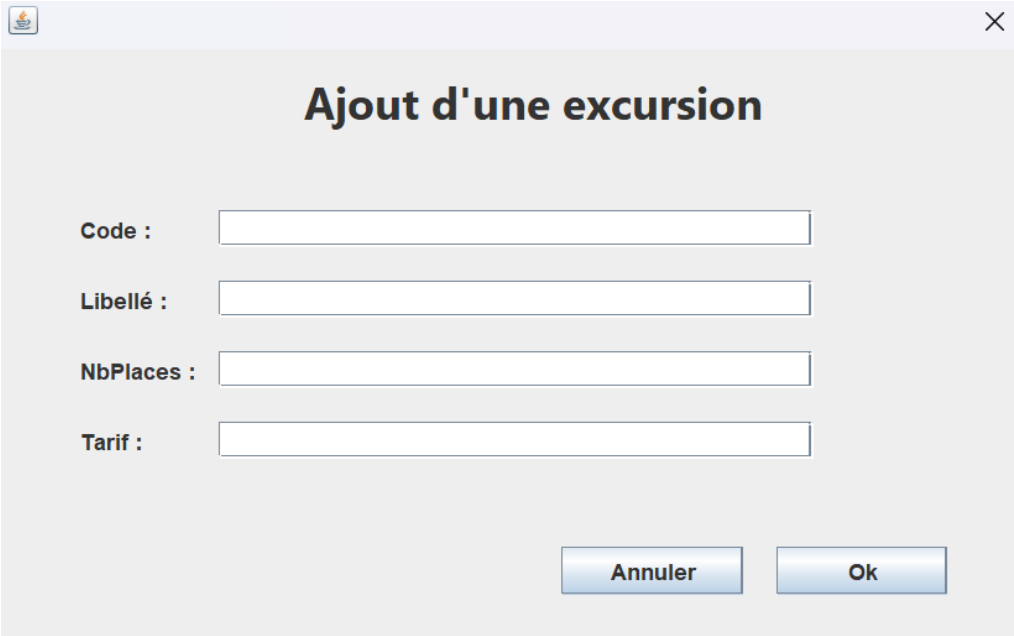

Figure 15 : Test n°1 : Fenêtre Ajoute d'une excursion.

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB
On entre les informations suivantes.

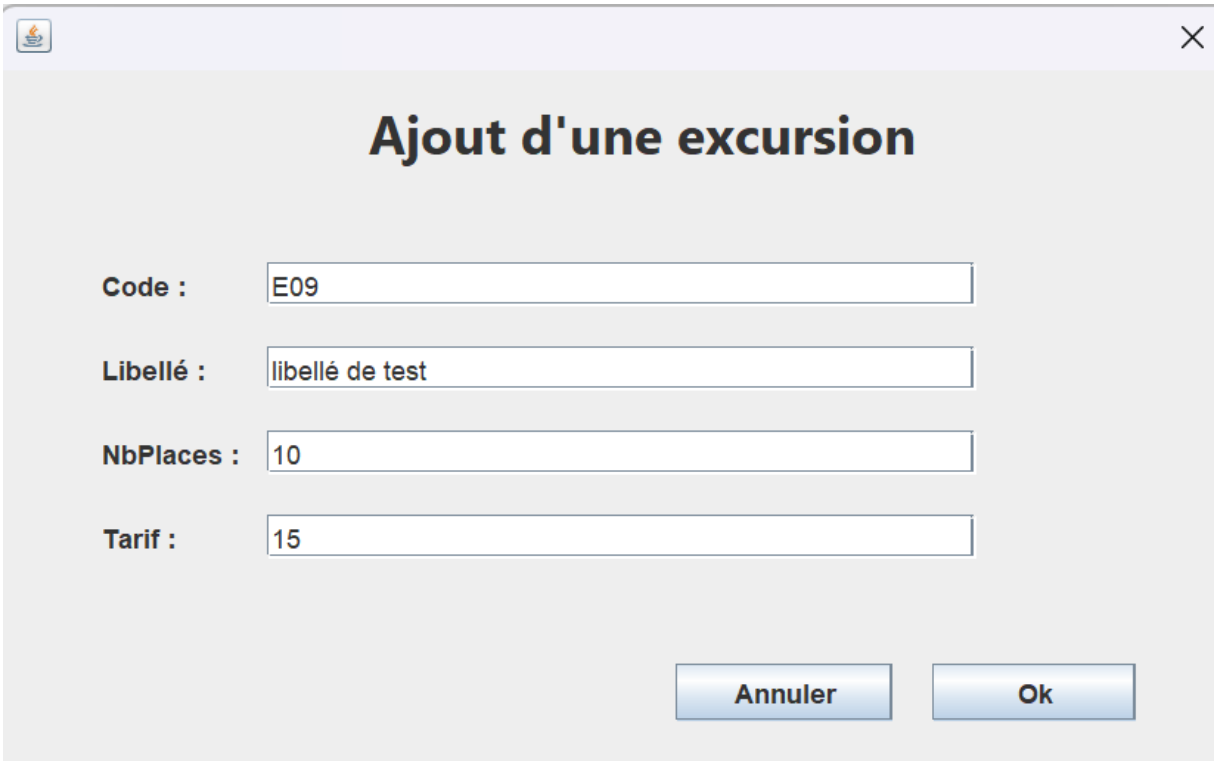

Figure 16 : Test n°1 : Champs de textes remplis.

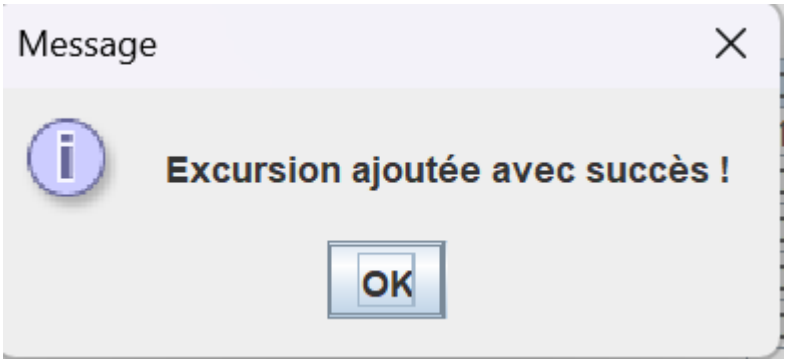

Figure 17 : Test n°1 : Excursion ajoutée avec succès.

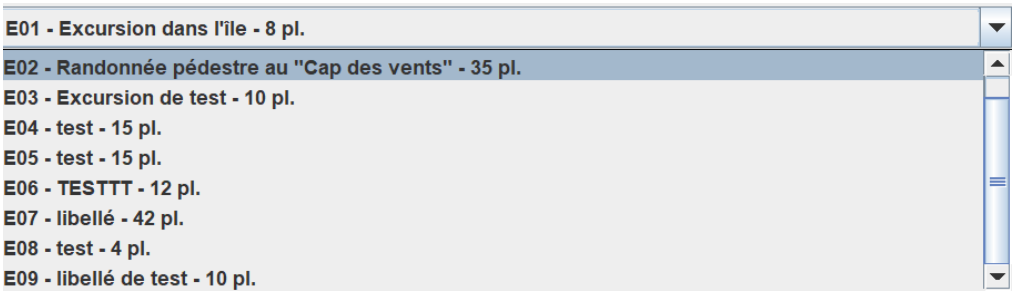

Figure 18 : : Test n°1 : Rafraichissement opéré avec succès.

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

Figure 19 : Test n°1 : L'excursion 9 a bien été ajoutée et ne possède aucune étape.

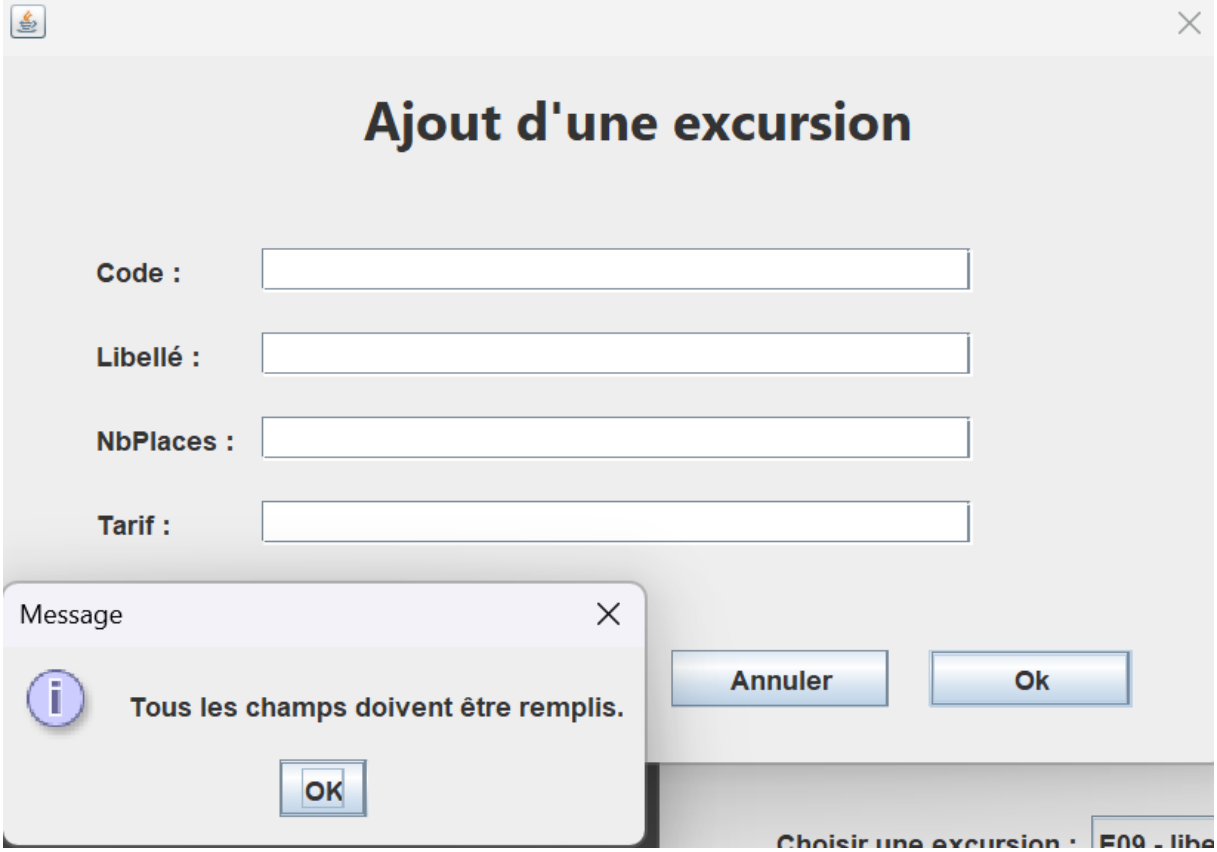

Figure 20 : Test n°2 : Message d'erreur : Les champs de textes sont vides.

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

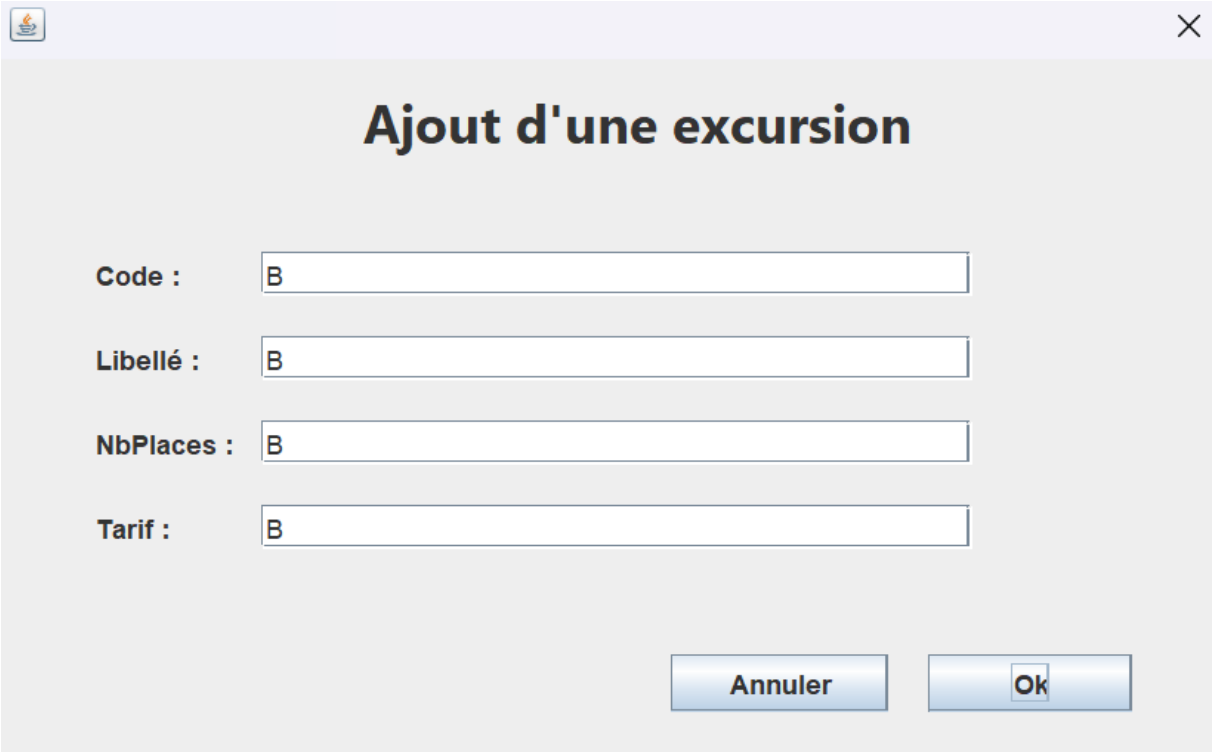

Figure 21 : Test n°2 : Tous les champs de texte sont remplis avec des caractères de type String.

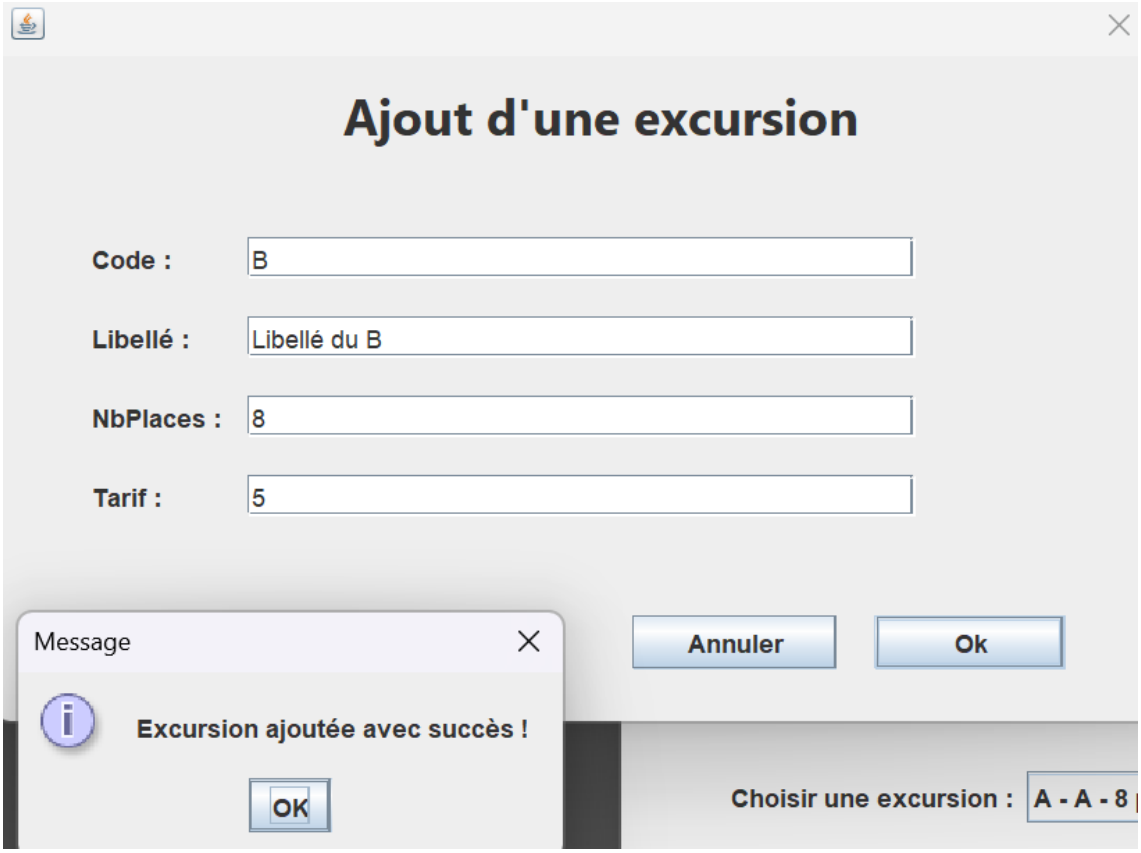

Figure 22 : Test n°2 : On remplace les informations String par des entiers quand c'est nécessaire. Puis, on clique sur « OK ». Un POP-UP apparaît bien pour indiquer que l'enregistrement est un succès.

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

Figure 23 : Test n°2 : La nouvelle excursion est bien ajoutée mais son numéro est différent des autres. En effet, la nomenclature est la suivante « EXX » ou E signifie excursion et X un chiffre.

DELAUNAY-GUITTON Benjamin
DE JESUS VIZCAINO Nikauly
1 SIOB

Bilan

Toutes les étapes ont été réalisées.

Aucun problème n’a été rencontré durant cette étape.